

UNIVERSIDAD DE EL SALVADOR

Facultad Multidisciplinaria De Occidente
Departamento de Ingeniería y Arquitectura

Ingeniería En Desarrollo De Software/Educación En Línea
LOGICA DE PROGRAMACION
SEMESTRE 1 / 2025



GUÍA DE LABORATORIO #4

Arreglos dinámicos, reglas de funcionamiento de la asignación de memoria, entrada y salidas por archivos.

Nombre: _____, Carnet: _____
Fecha: _____, Grupo teórico: _____

Objetivo: Conocer los conceptos y el uso de los arreglos dinámicos, asignación de memoria, entrada y salida por archivos, además de exponer una breve introducción de memoria dinámica y archivos C++.

Indicación general: Leer detenidamente cada una de las cuatro partes en que está dividida la actividad y brindar respuestas en las que se solicite.

Parte 1 - Introducción a la memoria dinámica en C++

Indicación: leer detenidamente el contenido de introducción a la memoria dinámica, en el lenguaje de programación C++.

La asignación dinámica de la memoria hace referencia a reservar un espacio en el montón (**heap**) para almacenar una información. Esta forma requiere el uso de la palabra reservada **new** para establecer el tamaño deseado del puntero:

```
int *p_numero = new int; // Espacio reservado dinámicamente
*p_numero = 55; // Asignación en el espacio dinámico
std::cout << *p_numero << std::endl; // 55
```

La vida de los datos en el montón (**heap**) existe mientras el programador no los elimine manualmente de la memoria, por ello cuando no necesitemos más la información deberemos

vaciar el espacio utilizando la palabra reservada **delete**, haciendo que el programa devuelva la memoria al sistema operativo:

```
delete p_numero; // Vaciamos el espacio reservado
```

En resumen, la memoria dinámica es un espacio de almacenamiento que se puede solicitar en tiempo de ejecución. Además de solicitar espacios de almacenamiento, también podemos liberarlos (en tiempo de ejecución) cuando dejemos de necesitarlos. Haciendo mas eficientes el uso de la memoria dentro de nuestros programas.

Parte 2 - Introducción a Archivos en C++

Indicación: leer detenidamente el contenido de introducción a la memoria dinámica, en el lenguaje de programación C++.

Ya se pueden manejar gran cantidad de datos del mismo y diferente tipo al mismo tiempo a través de arrays y estructuras. El problema es que el programa retiene los datos mientras esté ejecutándose y se pierden al terminar la ejecución. La solución para hacer que los datos no se pierdan es almacenarlos en archivo.

Los archivos son medios que facilita el lenguaje para almacenar los datos en forma permanente, normalmente en los dispositivos de almacenamiento estándar,

En C++ los archivos son la forma en la que C++ permite el acceso al disco, un archivo es simplemente un flujo externo: una secuencia de bytes almacenados en disco. Si el archivo se abre para salida, es un flujo de archivo de salida. Si el archivo se abre para entrada, es un flujo de archivo de entrada. Los tipos de archivos son:

- Archivos de texto.
- Archivos de entrada.
- Archivos de salida.
- Archivos Binarios

Parte 3 – Comentarios, Introducción a Objetos y Variables en C++

Indicación: leer detenidamente el contenido de introducción a la memoria dinámica, en el lenguaje de programación C++.

Un comentario es una nota legible por el programador que se inserta directamente en el código fuente del programa. Los comentarios son ignorados por el compilador y son para uso exclusivo del programador.

En C++ hay dos estilos diferentes de comentarios, los cuales tienen el mismo propósito: ayudar a los programadores a documentar el código de alguna manera.

- El símbolo `//` comienza un comentario de una sola línea de C++, que le dice al compilador que ignore todo, desde el símbolo `//` hasta el final de la línea. Por ejemplo

```
1 | std::cout << "Hello world!"; // Everything from here to the end of the line is ignored
```

- Comentarios de varias líneas, el par de símbolos `/*` y `*/` denota un comentario de varias líneas de estilo C. Todo lo que hay entre los símbolos se ignora.

```
1 | /* This is a multi-line comment.  
2 |    This line will be ignored.  
3 |    So will this one. */
```

Introducción a Objetos y Variables.

En C++, se desaconseja el acceso directo a la memoria. En cambio, accedemos a la memoria indirectamente a través de un objeto. Un objeto es una región de almacenamiento (generalmente memoria) que tiene un valor y otras propiedades asociadas.

Los objetos pueden tener o no nombre (anónimos). Un objeto con nombre se denomina variable y el nombre del objeto se denomina identificador. En nuestros programas, la mayoría de los objetos que creamos y usamos serán variables.

Definición de Variables.

Aquí hay un ejemplo de definición de una variable llamada `x`:

```
1 | int x; // define a variable named x, of type int
```

En tiempo de compilación, cuando el compilador ve esta declaración, se hace una nota de que estamos definiendo una variable, dándole el nombre `x`, y que es de tipo `int`, cada vez que el compilador vea el identificador `x`, sabrá que estamos haciendo referencia a esta variable.

En C++, el tipo de una variable debe conocerse en tiempo de compilación (cuando se compila el programa), y ese tipo no se puede cambiar sin volver a compilar el programa. Esto significa que una variable entera solo puede contener valores enteros. Si desea almacenar algún otro tipo de valor, deberá usar una variable diferente.

Parte 4- Desarrollar los siguientes puntos, ponderación de actividad 100%

Indicación: Brindar solución, desarrollar un programa si se amerita, a cada punto que se muestra a continuación, tomando como referencia los tópicos abordados en clase.

- a) Mencione la ventaja de utilizar arrays dinámicos en lugar de arrays estáticos.
- b) Ejemplifique y explique el uso de **ifstream**, **ofstream** y **fstream** a través de un ejemplo en código en C++.
- c) Escriba una definición propia y breve de los tipos de archivos expuestos en la parte 2 de esta guía.